

Isolation and Protection in Operating Systems

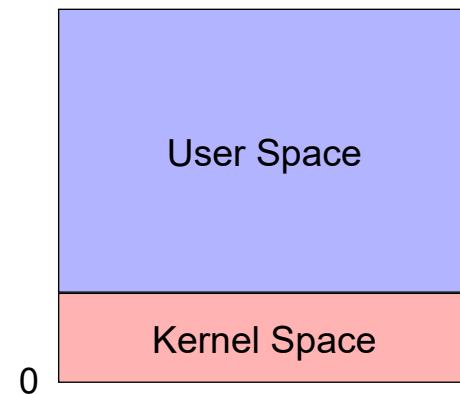
5. Kernel Isolation in x86-64

Prof. Dr. Michael Schöttner

Dr. Fabian Ruhland

Address Space Layout before Meltdown

- Each process has its own address space = *User Space*
- The kernel is mapped into every address space = *Kernel Space*
- Protection via segmentation = *Privilege Level*
- Protection via paging = Kernel is protected via U/S bit in page tables



- Discovered in July of 2017
- Affects CPUs from Intel, AMD and ARM
- Undetected for ~20 years
- What happens?
 - Bypasses isolation between operating system (kernel mode) and application (user mode)
 - Enables access to protected kernel memory from user mode and to the memory of other processes
- Naming: *Melting* hardware/operating system security barriers

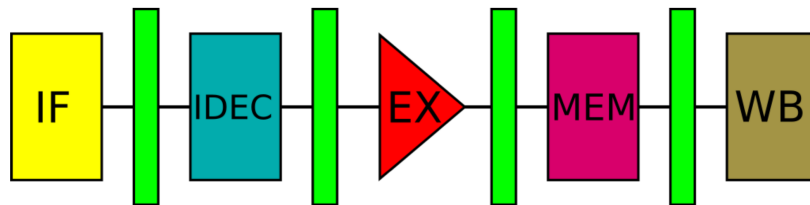


- Meltdown is a sidechannel attack
 - Attackers cannot directly access the data
 - Idea: Use independent events for communication (e.g. light on/off)
 - Extraction via a secret information channel (originates from embedded systems)
- Exploit is based on a combination of different acceleration functions in hardware and operating systems
 - Parallel and speculative execution of instructions
 - Caching and TLB
 - Address spaces

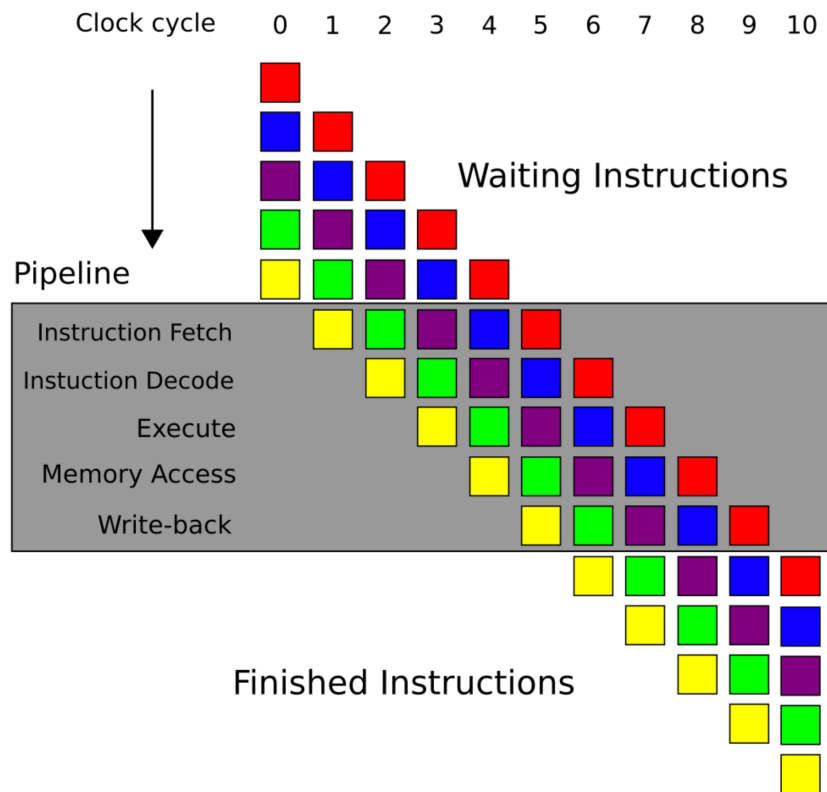
- Problem: Instructions are complex and accessing memory is slow

- Solution: Pipelining

- Decode instructions into micro operations (μ OPs) and load data beforehand
- Instruction execution workflow:
 1. Instruction Fetch (IF)
 2. Instruction Decode (IDEC)
 3. Instruction Execute (EX)
 4. Memory Access (MEM)
 5. Result Writeback (WB)



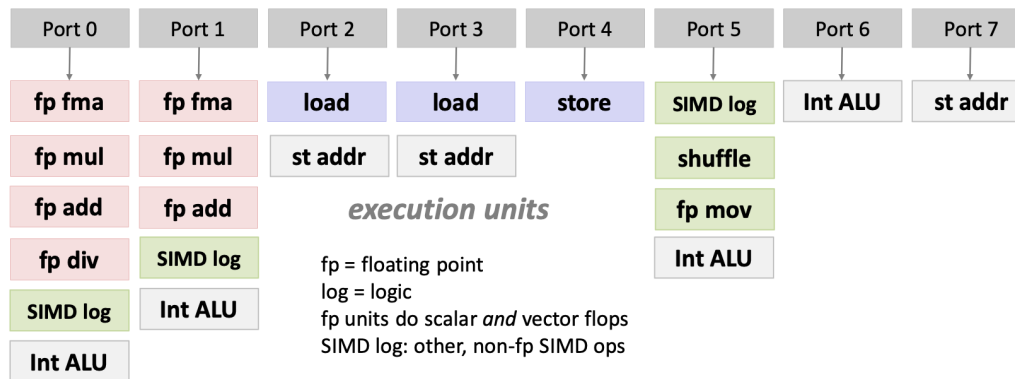
CPU: Pipelining



- Pipelining → Decode instructions and load data beforehand
- Problem: When the code branches, it is unclear where the execution path continues
- Solution: *Branch Prediction* and *Speculative Execution*
 - Changes done by executed code only become visible if the speculation was right
 - Works well with loops → High probability for another iteration

- Problem: Clock rates come near physical limits → Parallel execution for better scalability
- Most CPUs since 1998 are *superscalar*:
 - A superscalar CPU has multiple execution units
→ Can execute multiple instructions per clock

Execution Units and Ports (Skylake)



- Goal: use as all execution units if possible
- Distribute all sequential instructions of **one** thread on multiple execution units
→ *Instruction Level Parallelism* (ILP)
- x86 CPUs use a RISC-microarchitecture internally, decomposing assembly instructions into multiple μ OPs

- Example:

```
; Example 2.1. Out of order processing
mov eax, [mem1]
imul eax, 5
add eax, [mem2]
```

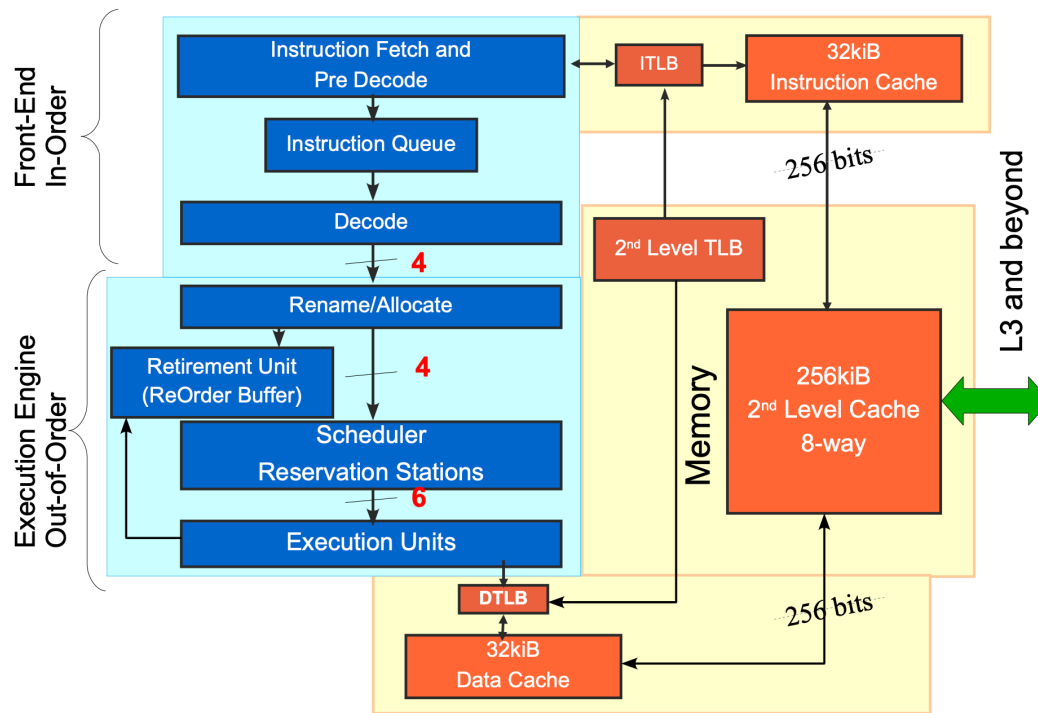
- ADD EAX, [MEM2] is decomposed into two μ OPs
- CPU fetches [MEM2] while imul is being executed

- *Register Renaming*: Registers are renamed by the microarchitecture
 - Internally a *Register Alias Table* (RAT) is used
- Example:
 - [MEM1] is multiplied by 6
 - [MEM3] is increased by 2
 - Both operations are independent from each other and can be executed out of order through automatic register renaming

```
; Example 2.3. Register renaming
1: mov eax, [mem1]
2: imul eax, 6
3: mov [mem2], eax
4: mov eax, [mem3]
5: add eax, 2
6: mov [mem4], eax
```

- At the end the changes done by the assembly instructions must become visible in the correct order (as dictated by the program) → *Retire in Order*
- This is done by the *Reorder Buffer* (ROB):
 - Monitors correct order of assembly instructions
 - Manages exception processing (in the right order)

Example: Nehalem Core Pipeline



High-level diagram of a Nehalem core pipeline, Michael E. Thomadakis, Ph.D., <https://progforperf.github.io/nehalem.pdf>

Example: Out-of-Order Execution

- Cache is completely replaced (e.g., by reading a very large array)
- Afterward the following program is executed:

```
1: *(volatile char*) 0; // raise_exception();  
2: array[84 * 4096] = 0; // access 84 page from array start
```

- Line 2 is obviously independent from line 1

Example: Out-of-Order Execution

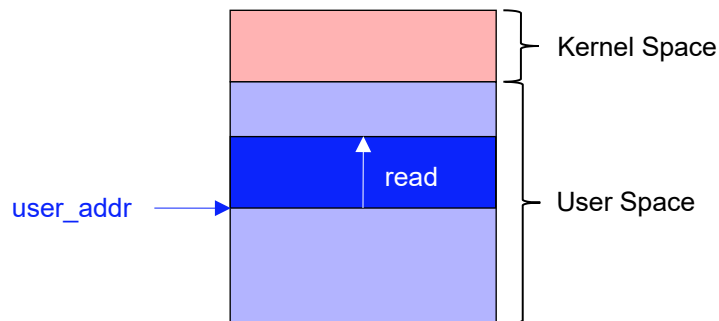
- Read the elements of array and measure the access times



- One access is much faster than the others
 - Instruction in line 2 has been executed
 - Exception (caused by line 1) was thrown afterward
 - Cache is not reset by the exception
 - We can indirectly observe the execution of a volatile instruction

Meltdown Attack (Principle)

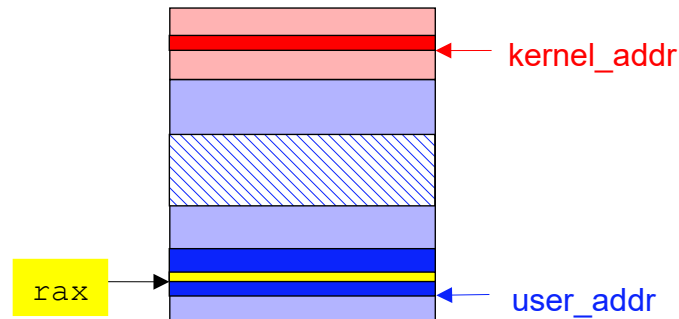
- Step 1: Delete the cache by allocating (and reading) a large array in user space (`user_addr` in the picture)



Meltdown Attack (Principle)

■ Step 2: Execute the following code

- Accessing `kernel_addr` is forbidden
- But accessing `user_addr` is allowed

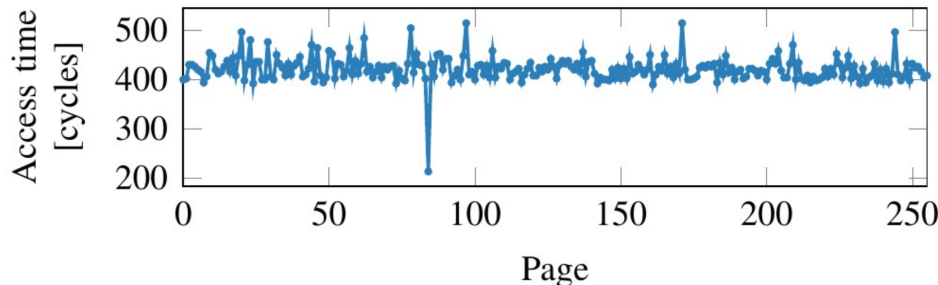


```
; user-mode code
```

```
1: mov rax, [kernel_addr]      ; access address in kernel space
2: and rax, 0xff               ; use lowest byte
3: mov rbx, [rax*128+user_addr] ; as index in an array in user-space
                               ; align the access with cache lines
                               ; here 128 bytes per cache line
```


Meltdown Attack (Principle)

- Step 3: Use a loop to read single bytes with a stride of 128 byte (size of a cache line), starting at `user_addr`
 - One access will be much faster than the others
 - This is exactly the cache line that was loaded by the **forbidden access**
 - This way, we can determine the number of the cache line and thus the index used to access the array → This is the **byte in `rax`**
 - We just read a byte from kernel space!
 - This can be continued indefinitely to read any memory from kernel space



- Step 4: Prevent the process from being terminated when the forbidden memory access is recognized
 - Simply register a signal handler for `SIGSEV`
 - This prevents termination of the process
 - Determine the index via time measurements (see step 3) inside the handler
 - Read the desired data from kernel space byte by byte

■ Why does this work?

- Lines 2 and 3 depend on line 1

```
1: mov rax, [kernel_addr]
2: and rax, 0xff
3: mov rbx, [rax*128+user_addr]
```

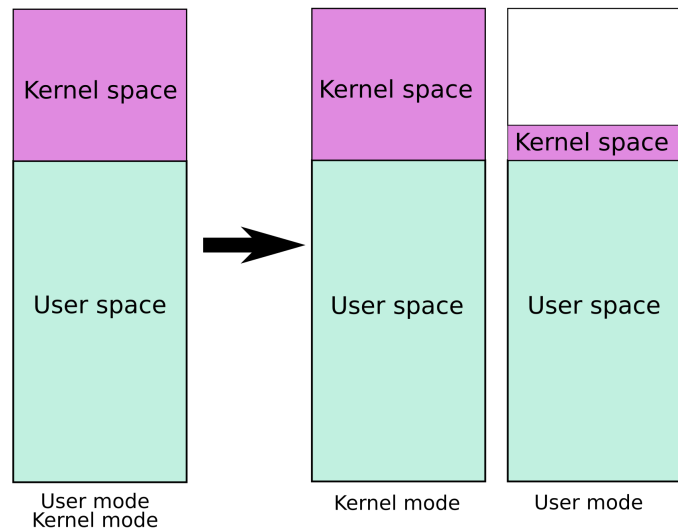
■ Speculative Execution

- The μ OPs for lines 2 and 3 are probably already waiting in the reservation station until data from line 1 is available
- As soon as the data from `[kernel_addr]` is available on the internal data bus, the waiting μ OPs are executed
- In parallel, the result from line 1 is being processed in the retirement unit, where the exception is thrown
- In the meantime (before the exception is thrown) the cache has already been filled for the μ OPs of line 3
- The exception resets the CPU state, but not the cache

- Most operating systems map the whole physical address space into the kernel space
- This allows attackers to read the whole memory via meltdown attacks
- Example: Private keys of NTFS filesystems were kept in kernel space memory and could be read via meltdown attacks
- `SIGSEV` is expensive, but on fast systems kernel memory can be read at about 500 KB/s
→ Highly critical security flaw

Solution: Kernel Page Table Isolation (KPTI)

- KPTI (formerly KAISER) realizes a fully separated address space for the Linux kernel →
 - When a user application is active, only a minimal portion of kernel code, needed to switch to the kernel space (e.g., for system calls), is mapped
 - During a system call the kernel is fully mapped
 - This is done by setting page directory entries
 - When returning from kernel code the TLB must be flushed
 - To make sure kernel pages are no longer accessible
 - This is expensive (up to 30% slower)



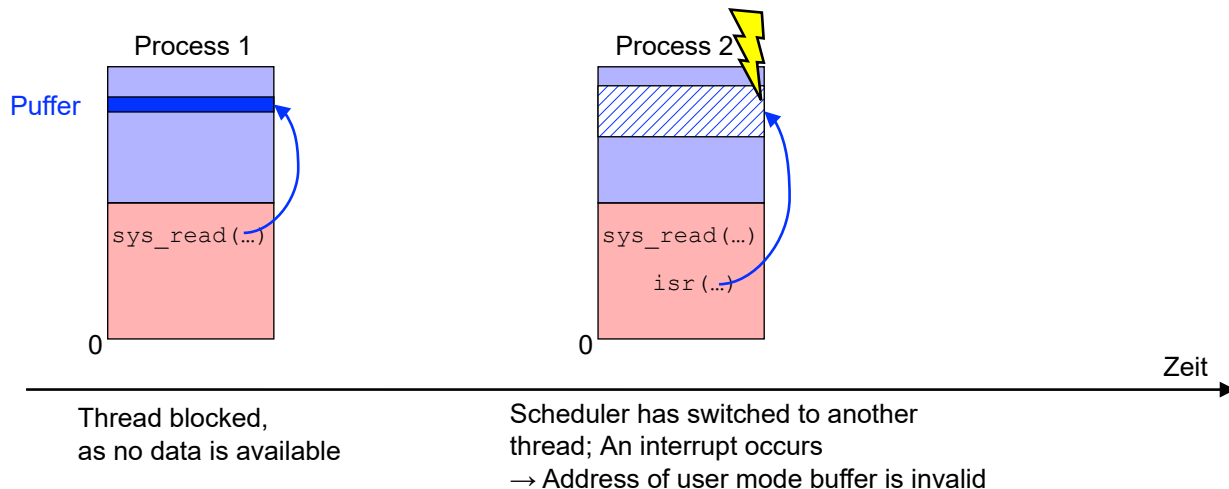
- Modern CPUs offer *Process Context Identifiers* (PCIDs)
 - 12 Bit → 4096 possible PCIDs
 - Must be activated in CR4
 - Current PCID is stored in bits 11:0 of CR3
- Allow the MMU to differentiate between TLB entries of multiple address spaces
- When accessing the TLB, only entries with the current PCID (in CR3) are used
- This way, returning from kernel code does not require flushing the whole TLB anymore!

- Lipp et.al., „Meltdown: Reading Kernel Memory from User Space“, USENIX Security Symposium, USA, 2018.
- Gruss et.al., „KASLR is Dead: Long Live KASLR“, Engineering Secure Software and Systems, Germany, 2017.
- Gruss et.al., „Kernel Isolation - From an Academic Idea to an Efficient Patch for Every Computer“, USENIX, 2018, Vol. 43, No. 4
- Comprehensive information on Meltdown and Spectre: <https://meltdownattack.com>

- Many system calls take a pointer to a buffer on the user heap as parameter
 - Reading: Input buffer (empty, for new data)
 - Writing: Output buffer (already filled with data)
- The kernel must allocate a same-sized buffer and copy the data (see next slides)
- Reason: At the time the user mode buffer is accessed, an other process may be active and the user mode addresses invalid

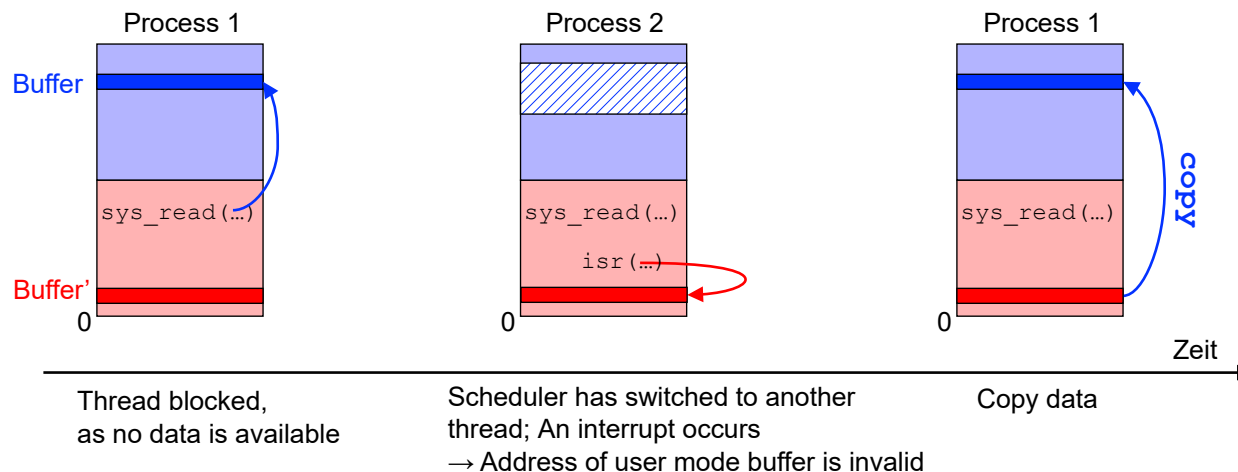
System Call: Reading

- If no data is available to read, the caller is blocked
- At a later point in time, an interrupt signals the device driver that new data is available
→ The waiting thread is deblocked
- At the time the interrupt occurs, another process may be active and the user mode addresses (i.e., the target buffer) are invalid



System Call: Reading

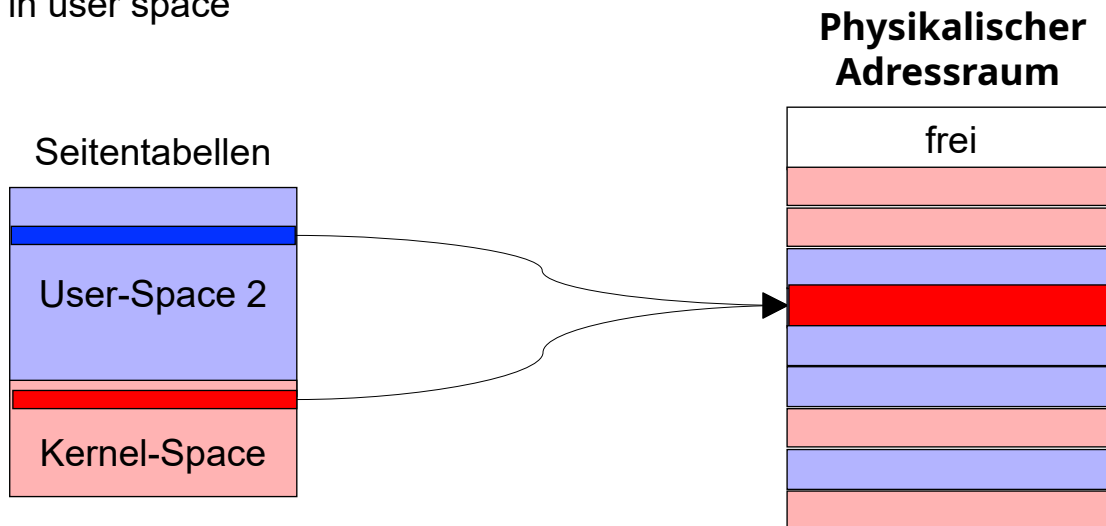
- If no data is available to read, the caller is blocked
- At a later point in time, an interrupt signals the device driver that new data is available
→ The waiting thread is deblocked
- At the time the interrupt occurs, another process may be active and the user mode addresses (i.e., the target buffer) are invalid



- As with reading, it may not be possible to write the data directly to the device
 - For example, the device may currently be busy with other data
- Because of that, the kernel also creates a buffer and copies the user data into it
- This way, it does not matter whether the process is still active when the target device is ready to receive data

■ Solution 1: Two mappings (UNIX/Linux)

- Drive implements the `mmap` function
- If an application calls `mmap` in the driver, it allocates a kernel space buffer and creates an additional mapping in user space



- Solution 2: Kernel uses physical addresses (Windows DirectIO)
 - I/O-Manager creates a *Memory Description List*:
 - A list of frames that is occupied by the buffer
 - The driver operates directly on the physical addresses
 - Usually the whole physical address space is mapped 1:1 in kernel space
 - This way, the address calculation is trivial (without page tables):
 - *Lower Half* kernel: physical address = virtual address
 - *Higher Half* kernel: physical address = virtual address - `kernel_offset`